Учебная практика: КТ2 (03.11), ИКБО-31 — памятка студентам

Срок сдачи

Вторая контрольная точка — Nov 3, 2026, 23:59. Сдача только через СДО, в задание «КТ2 — Аналитический этап».

Эта памятка — для группы **ИКБО-31**. Язык процедурной реализации на КТ2 вы **выбираете сами**: в индивидуальном задании она сформулирована как выполнение «средствами процедурного программирования», без указания языка. На КТ3 ту же задачу вы реализуете объектно-ориентированно — уже строго **на Джаве**, как записано в задании.

Сдаётся **один ZIP-архив**, как и на КТ1. Работы, загруженные отдельными файлами или в другом формате архива, к проверке не принимаются.

На КТ2 выбранный вами бизнес-процесс превращается в модели и в первую работающую программу. Всё, что сдаётся на этой точке, опирается на процесс, описанный на КТ1, поэтому начните с того, чтобы перечитать собственное описание и замечания по нему.

Состав архива

Внутри архива — строго такая структура. Имена папок и файлов менять нельзя: по ним работы проверяются.

```
Группа_ФамилияИО_КТ2.zip
├── report.docx              отчёт целиком: разделы КТ1 + аналитический
раздел
├── models/
|   ├── bpmn.bpmn            модель бизнес-процесса (исходник)
|   ├── bpmn.png             она же картинкой
|   ├── er_logical.drawio    логическая модель данных (исходник)
|   ├── er_logical.png
|   ├── algo_1.drawio        блок-схема первого алгоритма
|   ├── algo_1.png
|   ├── algo_2.drawio        блок-схема второго алгоритма
|   └── algo_2.png
├── src/
|   ├── README.md            как собрать, запустить и что делает программа
|   └── ...                  исходный код процедурной реализации
└── vcs/
    ├── git_log.txt          выгрузка истории коммитов
    └── repo_url.txt         ссылка на репозиторий
```

Каждая диаграмма сдаётся **парой**: исходник и `.png`. Исходник — тот формат, в котором вы рисовали: `.bpmn`, `.drawio` или `.puml`. Расширение в схеме выше — пример; имя файла до точки менять нельзя. Картинка без исходника не принимается: по исходнику проверяется, что диаграмма построена в нотации, а не нарисована фигурами.

Как части КТ2 связаны между собой

Четыре части КТ2 — не отдельные задания, а одна цепочка: каждая следующая выводится из предыдущей. При проверке сверяется именно эта связь.



как связаны части КТ2 · 5 артефактов

Если на диаграмме нет объекта данных, из которого выросла сущность, или в коде нет функции для операции из таблицы, цепочка разорвана — такие места возвращаются на доработку.

Модель бизнес-процесса в BPMN 2.0

Моделируется **тот же процесс, что описан текстом на КТ1**, — в нотации BPMN 2.0. Диаграмма и текстовое описание должны совпадать: те же участники, то же начальное событие, тот же результат.

Требования к диаграмме:

- Участники процесса — пулы и дорожки (pool, lane). Каждое действие стоит в дорожке того, кто его выполняет.
- Одно начальное и хотя бы одно конечное событие. Начальное — событие, с которого процесс стартует, конечное — результат из описания КТ1.
- Хотя бы один шлюз (gateway) с подписанными ветвями: процесс без единого ветвления обычно означает, что в нём пропущены проверки и отказы.
- Задачи названы глаголом и объектом: «Проверить комплектность заявки», а не «Проверка» или «Заявка».
- Документы и данные, которые передаются между шагами, показаны объектами данных (data object). Из них потом вырастает модель данных.
- Диаграмма читается без пояснений: не больше 15–20 задач на одном уровне. Если шагов больше, вынесите часть в подпроцесс.

Инструменты: bpmn.io и Camunda Modeler сохраняют настоящий `.bpmn`; draw.io тоже подходит, если использовать библиотеку фигур BPMN 2.0, а не обычные прямоугольники.

Автоматизируемые операции

Под диаграммой в отчёте вставить — таблицу операций, которые возьмёт на себя ваша программа. Это главный результат анализа процесса: он определяет, что вы будете программировать до конца практики.

№	Операция (задача на диаграмме)	Кто выполняет сейчас	Что будет делать система	Функция в коде
1	Проверить комплектность заявки	Менеджер вручную	Проверяет обязательные поля и сообщает, чего не хватает	`checkRequest()`

Строка выше — образец заполнения. Автоматизируемых операций должно быть не меньше трёх, и их названия должны дословно совпадать с задачами на диаграмме. Удобно выделить эти задачи на диаграмме цветом или пометкой. Последний столбец заполняется по вашей процедурной реализации.

Логическая модель данных

Логическая модель описывает, **какие данные нужны процессу**, — без привязки к конкретной СУБД. Типы столбцов, индексы и SQL появятся на КТ4, в физической модели.

Требования:

- Не меньше **5 связанных сущностей**. Каждая сущность должна встречаться в процессе: как участник, как объект данных на BPMN-диаграмме или как то, что создаётся или изменяется на одном из шагов.
- Хотя бы одна связь **многие-ко-многим**, разрешённая через ассоциативную сущность. Пример: «Заказ» и «Товар» связаны через «Позицию заказа» с количеством и ценой.
- У каждой сущности указан первичный ключ, у связей — мощность (1:1, 1:N, M:N) и внешние ключи.
- Модель приведена к **третьей нормальной форме**.

Нотацию выбирайте одну на всю диаграмму: «вороньи лапки» (crow's foot), IDEF1X или Чена. Смешивать не нужно.

В отчёте под диаграммой:

1. Таблица сущностей: название, назначение, атрибуты, первичный ключ, внешние ключи.
2. Описание связей: что с чем связано, мощность и смысл связи словами («у клиента может быть много заказов, каждый заказ принадлежит одному клиенту»).

3. Обоснование 3НФ: для каждой нормальной формы одно-два предложения, почему она выполняется. Покажите хотя бы один пример, где без нормализации возникло бы дублирование, и как оно устранено разбиением на сущности.

Алгоритмы

Нужно не меньше **двух алгоритмов**, которые реализуют автоматизируемые операции вашего процесса. Алгоритм ради алгоритма — сортировка пузырьком, не связанная с темой, — не засчитывается.

Требования:

- У двух алгоритмов **разные оценки сложности** по времени, например O(n) и O(n log n) или O(n·m).
- Хотя бы один алгоритм содержит ветвление и **вложенный цикл или рекурсию**.
- Простое добавление или удаление записи (CRUD) алгоритмом для этого пункта не считается: в нём нечего оценивать.

Примеры по разным темам: поиск свободного окна в расписании с учётом занятых интервалов; проверка заказа на конфликты с остатками склада; подбор исполнителя по нескольким критериям; построение маршрута по графу уровней или сцен; расчёт стоимости по тарифной сетке со скидками.

Блок-схемы

На каждый алгоритм — блок-схема по **ГОСТ 19.701-90**: ромб для решения с подписанными выходами «да» и «нет», параллелограмм для ввода-вывода, шестиугольник или пара символов границы цикла для циклов. Схема должна соответствовать коду: если в коде есть проверка, она есть и на схеме.

Оценка сложности

Для каждого алгоритма в отчёте:

1. Что такое n (и m, если параметров несколько) в вашей задаче: количество заказов, уровней, временных интервалов.
2. Оценка в O-нотации для **худшего случая** по времени.
3. Обоснование: сколько раз выполняется каждый цикл и почему. Одной фразы «сложность $O(n^2)$» недостаточно.
4. Оценка по памяти — желательно, одной строкой.

Если внутри алгоритма вызывается библиотечная функция — сортировка (`sorted()`, `Arrays.sort()`, `std::sort`), поиск в списке или словаре, — её сложность тоже учитывается в итоговой оценке.

Процедурная реализация

На КТ2 пишется первая версия программы — **в процедурном стиле**: только функции и стандартные структуры данных. На КТ3 эта же задача будет реализована объектно-ориентированно на Джаве, и две версии вы сравните между собой. Поэтому процедурная версия должна быть честно процедурной.

Выбор языка

Подойдёт любой язык, на котором можно писать в процедурном стиле: Си, C++, C#, Питон, Джава, Go и другие. Выбранный язык и его версию укажите в отчёте, в разделе о среде разработки, и в README. Если на КТ1 там был указан другой язык, исправьте.

Если процедурную версию писать на Джаве, на КТ3 вы будете сравнивать два стиля в одном языке. Если на другом языке — сравнение получится между двумя языками, и в нём нужно будет отдельно отметить, какие различия связаны с парадигмой, а какие с языком.

Что значит «процедурно»

На любом языке правило одно: программа состоит из функций, которые получают данные через параметры и возвращают результат. Своих классов для сущностей, методов у структур данных, интерфейсов и наследования нет — это ООП, оно на КТ3. Сущность из модели данных хранится как запись в стандартной структуре, а не как объект своего класса. Глобальных изменяемых переменных быть не должно.

Язык	Как писать процедурно	Чего не должно быть
Си, C++	Свободные функции; данные — в `struct` без методов, массивах, `std::vector`, `std::map`	Классов с методами, наследования, виртуальных функций
C#	Один `static class Program` со статическими методами; данные — в массивах, `List`, `Dictionary`	Своих классов и `record` для сущностей, интерфейсов

Язык	Как писать процедурно	Чего не должно быть
Джава	Один класс с `main` и статическими методами; данные — в массивах, `ArrayList`, `HashMap`	Своих классов, `record` и интерфейсов для сущностей
Питон	Только функции; данные — в `dict`, `list`, `tuple`, `set`	Своих классов, включая `@dataclass`, `NamedTuple`, `Enum`

Если выбранного языка нет в таблице, согласуйте его с руководителем до начала работы.

Что программа должна делать:

- Реализовать оба алгоритма из раздела выше и автоматизируемые операции из таблицы под BPMN-диаграммой.
- Работать с данными, структура которых соответствует логической модели: те же сущности и атрибуты. Хранить их можно в памяти или в файле CSV или JSON — база данных подключается только на КТ4.
- Запускаться из командной строки и показывать результат: консольного ввода-вывода достаточно, графический интерфейс не нужен.

В `src/README.md`:

- язык, его версия и команды сборки и запуска — например, `python main.py`, `javac` и `java`, `g++` и запуск программы, `dotnet run`;
- зависимости — если используются внешние библиотеки, они перечислены в файле менеджера пакетов (`requirements.txt`, `pom.xml`, `.csproj`, `CMakeLists.txt`), а не лежат в архиве готовыми файлами;
- пример входных данных и ожидаемого вывода, чтобы проверяющий мог запустить программу и сверить результат.

В отчёт код целиком не вставляется: достаточно фрагментов, реализующих алгоритмы, рядом с их блок-схемами. Полный код — в архиве и репозитории.

Отчёт, репозиторий и замечания по КТ1

Отчёт. Сдаётся тот же файл, что на КТ1, дополненный новым разделом — «Аналитический раздел» с четырьмя подразделами: модель бизнес-процесса, логическая модель данных, алгоритмы, процедурная реализация. Все диаграммы вставлены в отчёт как рисунки с подписями и упомянуты в тексте. Крупную диаграмму можно поместить на отдельную страницу альбомной ориентации или в приложение — главное, чтобы подписи читались.

Ограничений по объёму нет. Оформление — те же требования, что на КТ1: Times New Roman 14 пт, полуторный интервал, поля 30/10/20/20 мм.

Замечания по КТ1 исправляются в том же отчёте. Если описание процесса на КТ1 вернули на доработку, сначала исправьте его: диаграмма BPMN строится по исправленной версии.

Источники. Литература по BPMN, нормализации и оценке сложности добавляется в общий список и упоминается в тексте со ссылкой. Ограничение «не старше 5 лет» не распространяется на ГОСТ 19.701-90 и спецификацию BPMN 2.0 — это действующий стандарт и первоисточник нотации.

Репозиторий. Весь код КТ2 лежит в том же репозитории, что создан на КТ1. После сдачи КТ1 в нём должно появиться не меньше **3 новых коммитов в разные дни**: код, который весь целиком появился одним коммитом накануне сдачи, требованию не отвечает. В репозиторий стоит класть и исходники диаграмм. Результаты сборки — скомпилированные файлы и папки вроде `build/`, `bin/`, `target/`, `out/` — в репозиторий не кладутся: добавьте их в `.gitignore`. Файлы `vcs/git_log.txt` и `vcs/repo_url.txt` формируются так же, как на КТ1:

```
git log --all --date=iso --pretty=format:"%h %ad %an %s" > git_log.txt
```

Частые ошибки

Из-за этих ошибок работы чаще всего возвращаются на доработку.

Ошибка	Как должно быть
BPMN-диаграмма — обычная блок-схема из прямоугольников и стрелок, без пулов и дорожек	Участники — дорожки, действия стоят в дорожке исполнителя, есть события и шлюзы
На диаграмме другой процесс или другие участники, чем в тексте КТ1	Диаграмма и текст описывают одно и то же; если процесс уточнился, исправлен и текст
ER-диаграмма — это таблицы с типами `VARCHAR`, `INT`	Это физическая модель, она на КТ4. На КТ2 — сущности, атрибуты, ключи и связи
Сущности не встречаются в процессе («Пользователь», «Настройки», «Лог»)	Каждая сущность — участник, документ или объект, с которым работает процесс

Ошибка	Как должно быть
Связь многие-ко-многим нарисована, но не разрешена	Между сущностями стоит ассоциативная сущность со своими атрибутами
Алгоритм не связан с темой или сводится к добавлению записи	Алгоритм реализует операцию из таблицы автоматизируемых операций
«Сложность O(n²)» без объяснения	Сказано, что такое n, сколько раз выполняется каждый цикл и почему
Блок-схема не совпадает с кодом	Каждая проверка и каждый цикл кода есть на схеме
В процедурной версии свои классы для сущностей (`Order`, `Client`) — это уже ООП, оно на КТ3	Только функции и стандартные структуры данных; в Джаве и C# — один класс-обёртка со статическими методами
Программа не собирается или не запускается по инструкции из README	Проверьте сборку и запуск на чистой копии репозитория перед сдачей

Итог контрольной точки

Результат КТ2 — **отметка о выполнении аналитического этапа в рабочем графике практики**. Она ставится руководителем, если сданный материал отвечает требованиям выше.

К этому моменту у вас есть процесс в нотации с выделенными операциями для автоматизации, модель данных, которую на КТ4 вы перенесёте в СУБД, и первая работающая программа, которую на КТ3 вы перепишете в объектно-ориентированном стиле.

Проверьте перед отправкой

Это минимум для зачёта этапа. Если хотя бы один пункт не выполнен, работа возвращается на доработку без разбора содержания.

- [ ] Архив назван по образцу `Группа_ФамилияИО_КТ2.zip` и содержит ровно ту структуру папок, что указана выше
- [ ] Каждая диаграмма есть в двух файлах: исходник и `.png`
- [ ] BPMN-диаграмма описывает процесс из КТ1: пулы и дорожки, начальное и конечное события, хотя бы один шлюз

- [ ] Под диаграммой — таблица из не меньше чем 3 автоматизируемых операций, названия совпадают с задачами на диаграмме
- [ ] В логической модели не меньше 5 сущностей, есть разрешённая связь многие-ко-многим, обосновано приведение к 3НФ
- [ ] Не меньше 2 алгоритмов с разной сложностью, хотя бы в одном ветвление и вложенный цикл или рекурсия
- [ ] На каждый алгоритм — блок-схема по ГОСТ 19.701-90 и обоснованная оценка сложности в худшем случае
- [ ] Программа написана в процедурном стиле, язык и его версия указаны в README и отчёте, реализует оба алгоритма, собирается и запускается по инструкции из `src/README.md`
- [ ] После КТ1 в репозитории не меньше 3 новых коммитов в разные дни, `git_log.txt` и `repo_url.txt` на месте

Что будет дальше

КТ3: диаграмма классов и объектно-ориентированная реализация на Джаве, сопоставление с процедурной версией, состав команды, взаимное рецензирование кода, анализ рынка труда.